

Orden de construcción desde cero del flujo de Clientes

Guía paso a paso — archivo por archivo, capa por capa — de cómo se construiría el flujo de `GET /api/clientes` y `GET /api/clientes/{id}` si el proyecto estuviera vacío hoy.

Orden de construcción desde cero — Flujo “Consultar Clientes”

Este documento responde a una pregunta distinta a la del análisis anterior: **no** “¿cómo funciona lo que ya existe?”, sino **“si Vitamedica.Microservicios.Clientes estuviera vacío hoy, en qué orden exacto construiría, archivo por archivo, todo el flujo de GET /api/clientes y GET /api/clientes/{id} , y por qué en ese orden y en ese lugar.”**

La regla que gobierna todo el orden es una sola, y conviene tenerla siempre presente:

Se construye de adentro hacia afuera: primero lo que no depende de nada (Domain), y al final lo que depende de todo (Api). Cada capa nueva solo puede compilar si la anterior ya existe. El compilador de C# te “obliga” a seguir este orden — si intentas saltarte un paso, el archivo que estás escribiendo no compila porque le falta algo de una capa inferior.



Con eso en mente, aquí está la secuencia completa, paso a paso.

PASO 0 — Antes de escribir código: la solución y los proyectos vacíos

Antes de cualquier archivo `.cs` , tiene que existir el esqueleto de proyectos, porque cada archivo de código vive dentro de un proyecto (`.csproj`) concreto, y los `.csproj` son los que declaran quién puede referenciar a quién.

1. `Vitamedica.Microservicios.Clientes.slnx` — el archivo de solución. Sin esto no hay “proyecto” que abrir en el IDE, es solo el contenedor.
2. `src/Vitamedica.Microservicios.Clientes.Domain/Vitamedica.Microservicios.Clientes.Domain.csproj` — proyecto de clases (`Microsoft.NET.Sdk`), **sin ninguna referencia** a otro proyecto ni paquete externo (aparte de `InternalsVisibleTo` hacia el proyecto de tests, que se agrega después).
3. `src/Vitamedica.Microservicios.Clientes.Application/Vitamedica.Microservicios.Clientes.Application.csproj` — proyecto de clases, con `ProjectReference` a `Domain.csproj` .
4. `src/Vitamedica.Microservicios.Clientes.Infrastructure/Vitamedica.Microservicios.Clientes.Infrastructure.csproj` — proyecto de clases, con `ProjectReference` a `Domain.csproj` (y hoy también a `Application.csproj` , aunque como vimos en el análisis anterior esa segunda referencia no llega a usarse). Necesita además los paquetes de EF Core (`Microsoft.EntityFrameworkCore` , `...SqlServer` , `...Design`).
5. `src/Vitamedica.Microservicios.Clientes.Api/Vitamedica.Microservicios.Clientes.Api.csproj` — proyecto web (`Microsoft.NET.Sdk.Web`), con `ProjectReference` a `Application.csproj` e `Infrastructure.csproj` , y el paquete `Swashbuckle.AspNetCore` para Swagger.

¿Por qué en este orden y no otro? Porque un `.csproj` con una `ProjectReference` a otro proyecto que todavía no existe no compila. Domain no referencia a nadie, así que puede (y debe) crearse primero. Cada proyecto siguiente ya “sabe” a quién va a referenciar antes de que exista ni un solo archivo `.cs` dentro.

CAPA 1 — DOMAIN (el núcleo, sin dependencias)

Carpeta: `src/Vitamedica.Microservicios.Clientes.Domain/`

Esta es la primera capa con código real porque es la única que no necesita que exista nada más. Todo lo demás (Application, Infrastructure, Api) va a depender de lo que se defina aquí, así que tiene que quedar listo antes.

Paso 1.1 — La entidad: `Domain/Entities/Cliente.cs`

Qué es: la clase `Cliente` , con sus propiedades (`Id` , `Nombre` , `ApellidoPaterno` , `ApellidoMaterno` , `FechaNacimiento` , `GeneroId` , `Rfc` , `Curp`) y la propiedad calculada `NombreCompleto` .

Por qué se crea primero, antes que cualquier otra cosa: es el vocabulario básico del negocio. No puedes escribir un repositorio, un DTO, ni un handler si todavía no existe la “cosa” (`Cliente`) sobre la que todos esos elementos van a trabajar. Es literalmente el sustantivo del que dependen todos los verbos (consultar, mapear, listar) que se agregan después.

Por qué en esta carpeta y no en otra: `Domain/Entities/` es donde viven los modelos de negocio puros — clases que **no importan nada de EF Core, ni de ASP.NET, ni de ningún framework**. Esto es intencional: si mañana cambia la forma de guardar los datos (de SQL Server a una API externa, por ejemplo), `Cliente` no debería tener que cambiar. Por eso esta clase no tiene ni un `using` de Entity Framework.

Decisiones de diseño a replicar al construirla:

- Setters `private` , para que nadie fuera de la clase pueda mutarla libremente (inmutabilidad hacia afuera).
- Un constructor `private` sin parámetros — EF Core lo necesita internamente para materializar objetos desde la base de datos vía reflection, pero no debe ser usable desde código de aplicación.
- Un constructor `internal` con todos los parámetros, habilitado para pruebas unitarias (ver Paso 1.4).
- Ninguna referencia a `Microsoft.EntityFrameworkCore` ni a ningún paquete externo.

Con quién se relaciona (todavía no, pero se relacionará): con `IClienteRepository` (Paso 1.3, la va a devolver), con `ClienteConfiguration` en Infrastructure (Paso 3.2, la va a mapear a una tabla), y con `ClienteDto` / `ClienteMapper` en Application (Pasos 2.1–2.2, la van a convertir en DTO). Ahora mismo, en este paso, `Cliente` no conoce a ninguno de ellos — la relación es de “afuera hacia adentro”: son los otros los que la conocen a ella, nunca al revés.

Paso 1.2 — La excepción de dominio: `Domain/Exceptions/DomainException.cs`

Qué es: una clase `DomainException : Exception` para representar violaciones de reglas de negocio.

Por qué se crea aquí, en este momento: aunque en este flujo de solo lectura nunca se llegue a lanzar (como vimos en el análisis anterior, hoy es una pieza preparada pero sin uso), conviene definirla junto con la entidad porque es parte del vocabulario del dominio: es el tipo de error que las reglas de negocio de `Domain` usarían para señalar que algo viola una invariante. Si más adelante `Cliente` (o una entidad nueva) necesita validar algo en su constructor, ya existe el tipo correcto de excepción para lanzar, sin tener que decidir esto a mitad de otra tarea.

Por qué en `Domain/Exceptions/` y no en Application: porque una regla de negocio (“un asegurado no puede tener una fecha de nacimiento futura”, por ejemplo) es una regla del dominio, no un detalle de cómo se orquesta un caso de uso. Application deberá saber **atrapar** este tipo de excepción (eso pasa más adelante, en el diseño de `Result`), pero quien la **lanza** debe ser Domain.

Relación: ninguna dependencia saliente. Será usada (potencialmente) por `Cliente` y capturada por los handlers de Application.

Paso 1.3 — El contrato de acceso a datos: `Domain/Repositories/IClienteRepository.cs`

Qué es: la interfaz `IClienteRepository` con los métodos `ObtenerPorIdAsync(int id, CancellationToken ct)` y `ObtenerTodosAsync(CancellationToken ct)` .

Por qué se crea en este momento (después de la entidad, antes de todo lo demás): ya existe `Cliente` (Paso 1.1), así que ya se puede escribir una interfaz que hable de “traer un `Cliente` ” o “traer varios `Cliente` ”. Esta interfaz es la pieza que hace posible que Application (capa 2) pueda escribirse **sin saber todavía cómo se va a implementar realmente el acceso a datos** — ese “cómo” (EF Core, SQL Server) llega recién en la capa 3 (Infrastructure).

Por qué en `Domain/Repositories/` y no en Infrastructure: esta es la decisión arquitectónica más importante de todo el proyecto, y vale la pena entenderla bien: **el contrato vive donde vive el consumidor conceptual (el negocio), no donde vive la implementación técnica**. Domain (y luego Application) necesitan poder “pedir un Cliente” sin saber si detrás hay SQL Server, una API REST, un archivo Excel o una caché en memoria. Si la interfaz viviera en Infrastructure, Application tendría que referenciar a Infrastructure para poder usarla — y eso invertiría la dependencia al revés de lo que se busca (Application terminaría “sabiendo” de EF Core indirectamente). Al vivir en Domain, es Infrastructure quien depende de Domain para poder implementar el contrato — nunca al revés. Esto es el principio de **Inversión de Dependencias** (la “D” de SOLID) aplicado literalmente, no en teoría.

Por qué es de solo lectura (sin `Agregar` / `Actualizar`): porque el propósito de este microservicio, decidido de entrada, es leer datos que pertenecen a otro sistema. La interfaz refleja exactamente lo que el negocio necesita — ni más ni menos. Si se define un método que nadie va a implementar todavía (como `Agregar`), se estaría rompiendo YAGNI (You Aren’t Gonna Need It) sin necesidad real.

Con quién se relaciona:

- Depende de `Cliente` (Paso 1.1) — está en su firma (`Task<Cliente?>` , `Task<IReadOnlyList<Cliente>>`).
- Será **implementada** por `ClienteRepository` en Infrastructure (Paso 3.3) — esa es la única capa que puede depender de Domain e “cerrar el contrato”.
- Será **consumida** (inyectada) por los handlers de Application (Pasos 2.5 y 2.6) — Application solo conoce la interfaz, nunca la implementación concreta.

Paso 1.4 — (En paralelo o justo después) El proyecto de tests:

`tests/Vitamedica.Microservicios.Clientes.Domain.Tests/`

Aunque técnicamente es “otro proyecto”, conviene crearlo inmediatamente después de la entidad, no al final, porque valida que `Cliente` se comporta como se espera **antes** de que el resto de las capas empiecen a depender de ella.

- 1. Agregar `<InternalsVisibleTo Include="Vitamedica.Microservicios.Clientes.Domain.Tests" />` al `.csproj` de Domain — esto es lo que permite que el proyecto de tests use el constructor `internal` de `Cliente` sin exponerlo públicamente al resto del mundo.
- 2. Crear `tests/.../ClienteTests.cs` con pruebas del cálculo de `NombreCompleto` y de la correcta asignación de propiedades (exactamente el patrón que ya usa `ClienteTests.cs`).

Por qué ahora y no al final del proyecto: validar el dominio temprano evita que un error de lógica (por ejemplo, en el cálculo de `NombreCompleto`) se arrastre silenciosamente hasta capas superiores que ya lo están usando. Es mucho más barato corregirlo aquí que después de que Application, Infrastructure y Api ya lo consumen.

CAPA 2 — APPLICATION (los casos de uso)

Carpeta: `src/Vitamedica.Microservicios.Clientes.Application/`

Ahora que existe el “vocabulario” de Domain (`Cliente`, `IClienteRepository`), se puede construir la capa que orquesta **qué hace el sistema** con ese vocabulario, sin saber todavía **cómo llega la petición** (eso es Api) ni **cómo se accede realmente a los datos** (eso es Infrastructure).

Paso 2.1 — La forma de salida: `Application/Dtos/ClienteDto.cs`

Qué es: el `record ClienteDto(int Id, string NombreCompleto, DateTime FechaNacimiento, byte GeneroId, string? Rfc, string? Curp)`.

Por qué se crea antes que los handlers: un handler necesita saber qué va a devolver antes de poder escribirse. El DTO define ese “contrato de salida” hacia el mundo exterior.

Por qué existe un DTO en vez de devolver `Cliente` directamente: para desacoplar la forma en que el negocio modela a `Cliente` (con setters privados, constructores internos, pensada para reglas de negocio) de la forma en que se serializa hacia HTTP/JSON. Si `Cliente` cambiara su forma interna (por ejemplo, se le agrega una propiedad de auditoría que no debe salir por la API), el DTO sigue exponiendo solo lo que se decide exponer. Es además lo que evita el problema de “mass assignment” si algún día existieran endpoints de escritura: nunca se bindea directamente contra la entidad de dominio.

Por qué en `Application/Dtos/` y no en Domain ni en Api: no es un concepto de negocio (Domain no debería preocuparse de cómo se ve algo en JSON), y tampoco es un detalle de transporte HTTP puro (Api no debería decidir la forma de los datos, solo transportarlos) — es una decisión de “qué expone este caso de uso hacia quien lo consuma”, que es exactamente responsabilidad de Application.

Relación: no depende de nada (es un `record` con tipos primitivos). Será producido por `ClienteMapper` (Paso 2.2) y consumido por Api (Paso 4.2).

Paso 2.2 — El mapeo: `Application/Clientes/ClienteMapper.cs`

Qué es: el método de extensión `ADto(this Cliente cliente)` que convierte una entidad `Cliente` en un `ClienteDto`.

Por qué justo después del DTO: ya existen ambos lados de la conversión (`Cliente` de Domain, `ClienteDto` de Application), así que se puede escribir la función que traduce de uno a otro.

Por qué en `Application/Clientes/` (carpeta de feature) y no en `Common/`: porque este mapeo es específico del feature “Clientes” — si el proyecto crece con más entidades, cada una tendrá su propio mapper junto a sus propios casos de uso, en vez de acumular todos los mappers en una carpeta genérica que crecería sin límite y perdería cohesión.

Relación: depende de `Cliente` (Domain) y de `ClienteDto` (Application, mismo proyecto). Será usado por los handlers (Pasos 2.5 y 2.6).

Paso 2.3 — Las abstracciones comunes: `Application/Common/Abstractions/IQueryHandler.cs` y `Result.cs`

Qué son: - `IQueryHandler<TQuery, TResult>` — el contrato genérico que todo “caso de uso de consulta” va a implementar (`Task<TResult> HandleAsync(TQuery query, CancellationToken ct)`). - `Result` / `Result<T>` — el tipo que representa éxito/fallo sin usar excepciones para control de flujo.

Por qué se crean antes de escribir el primer caso de uso concreto: son la “forma” que van a tener todos los casos de uso. Definir primero la forma genérica (una sola vez) y después escribir los casos concretos siguiendo esa forma es más ordenado que ir descubriendo el patrón caso por caso. Además, `Result<T>` es necesario porque `ObtenerClientePorIdQueryHandler` (Paso 2.6) lo va a devolver como tipo de retorno.

Por qué en `Application/Common/Abstractions/` y no repetido en cada handler: porque es infraestructura *del propio patrón* de Application, transversal a todos los casos de uso presentes y futuros — no pertenece a ningún feature en particular (“Clientes”, u otro que se agregue después), sino a la forma en que **todos** los features se van a escribir.

Por qué no se usó una librería externa (MediatR) para esto: con solo 2 casos de uso, escribir esta abstracción mínima a mano evita agregar una dependencia externa (y su curva de aprendizaje) que no aporta valor real todavía. Es una aplicación consciente de YAGNI.

Relación: `IQueryHandler<TQuery, TResult>` no depende de nada concreto (es genérico). `Result` / `Result<T>` tampoco depende de nada. Ambos serán implementados/usados por los handlers concretos (Pasos 2.5 y 2.6).

Paso 2.4 — Los “Query” (la intención de consulta): `Application/Cientes/Queries/ListarCientesQuery.cs` y `ObtenerClientePorIdQuery.cs`

Qué son: `public record ListarCientesQuery;` y `public record ObtenerClientePorIdQuery(int Id);` — objetos inmutables que representan “quiero listar todos los clientes” y “quiero obtener el cliente con este id”, respectivamente.

Por qué se crean antes que sus handlers: un handler siempre implementa `IQueryHandler<TQuery, TResult>` para un `TQuery` concreto — necesita que ese tipo ya exista para poder declarar su propia firma.

Por qué son `record` separados de sus handlers, y no simplemente parámetros sueltos del método: al ser un objeto con nombre propio, la intención queda explícita y es fácil de extender más adelante (por ejemplo, agregar un filtro de fecha a `ListarCientesQuery` sin tocar la firma del handler en su forma general — solo se le agregan propiedades al record).

Relación: no dependen de nada más que de tipos primitivos (`int`). Son el parámetro de entrada de sus respectivos handlers.

Paso 2.5 — El primer handler: `Application/Cientes/Queries/ListarCientesQueryHandler.cs`

Qué es: la clase `ListarCientesQueryHandler : IQueryHandler<ListarCientesQuery, IReadOnlyList<ClienteDto>>`, que recibe `IClienteRepository` por constructor y, en `HandleAsync`, llama a `_repository.ObtenerTodosAsync(ct)` y mapea el resultado con `.Select(c => c.ADto()).ToList()`.

Por qué se puede escribir recién ahora, y no antes: necesita **todas** las piezas anteriores ya existentes: `IClienteRepository` (Domain, Paso 1.3) para poder recibirla por constructor, `ListarCientesQuery` (Paso 2.4) como su tipo de entrada, `ClienteDto` y `ClienteMapper.ADto()` (Pasos 2.1–2.2) para producir la salida, e `IQueryHandler<,>` (Paso 2.3) como contrato a implementar. Este handler es literalmente el punto donde convergen todas las piezas construidas hasta ahora.

Con quién se relaciona: recibe `IClienteRepository` por inyección de dependencias (todavía sin implementación concreta — eso llega en Infraestructure); usa `ClienteMapper`; implementa `IQueryHandler`. Será, a su vez, inyectado en el Controller (Paso 4.2).

Paso 2.6 — El segundo handler: `Application/Cientes/Queries/ObtenerClientePorIdQueryHandler.cs`

Qué es: la clase `ObtenerClientePorIdQueryHandler : IQueryHandler<ObtenerClientePorIdQuery, Result<ClienteDto>>`, que llama a `_repository.ObtenerPorIdAsync(query.Id, ct)` y devuelve `Result<ClienteDto>.Fallido(...)` si es `null`, o `Result<ClienteDto>.Exitoso(cliente.ADto())` si lo encuentra.

Por qué después del primero y no antes o en paralelo: no hay una dependencia técnica estricta entre ambos handlers (podrían haberse escrito en el orden inverso), pero conviene escribir primero el caso más simple (`Listar`, sin parámetros, sin manejo de “no encontrado”) y luego el que introduce un concepto nuevo (`Result<T>` para el caso de fallo). Es más fácil razonar sobre `Result<T>` cuando ya está claro y probado el flujo “feliz” del otro caso de uso.

Relación: las mismas que el handler anterior, más el uso explícito de `Result<T>` (Paso 2.3) para modelar el caso “no existe el cliente con ese id”.

Paso 2.7 — El registro en el contenedor de dependencias: `Application/DependencyInjection.cs`

Qué es: el método de extensión `AddApplication(this IServiceCollection services)` que registra `services.AddScoped<ObtenerClientePorIdQueryHandler>()` y `services.AddScoped<ListarCientesQueryHandler>()`.

Por qué es el último archivo de esta capa: solo se puede registrar en el contenedor de DI algo que ya existe. Este archivo es, en esencia, el “índice” de todo lo que Application ofrece hacia afuera — tiene que escribirse después de que los handlers ya estén definidos.

Por qué este método vive en Application y no se registra directamente en `Program.cs` de Api: para que Api no necesite saber los nombres concretos de cada handler que Application decide exponer — Api solo llama a `builder.Services.AddApplication()` y confía en que Application se registra a sí mismo correctamente. Esto mantiene a Api desacoplada de los detalles internos de Application (si mañana se agregan 10 handlers más, `Program.cs` no cambia ni una línea).

Relación: depende de todos los handlers de esta capa. Será invocado desde `Program.cs` (Paso 4.1).

CAPA 3 — INFRASTRUCTURE (el acceso real a datos)

Carpeta: `src/Vitamedica.Microservicios.Cientes.Infraestructure/`

Domain ya definió *qué* se necesita (`IClienteRepository`) y Application ya definió *cómo se usa* ese contrato dentro de un caso de uso. Ahora toca la única capa que sabe que, detrás de todo esto, hay una base de datos SQL Server real con una tabla física ya existente.

Paso 3.1 — El `DbContext` : `Infrastructure/Persistence/AppDbContext.cs`

Qué es: la clase `AppDbContext : DbContext`, con `DbSet<Cliente> Clientes`, configurada con `QueryTrackingBehavior.NoTracking` por defecto y que carga sus configuraciones vía `ApplyConfigurationsFromAssembly`.

Por qué se crea primero dentro de esta capa, antes del mapeo de columnas y del repositorio: es la “puerta de entrada” a EF Core — tanto la configuración de columnas (Paso 3.2) como el repositorio (Paso 3.3) dependen de que exista un `DbContext` con un `DbSet<Cliente>` ya declarado.

Por qué `NoTracking` por defecto: porque este microservicio, de punta a punta, nunca escribe — solo lee. Activar el change tracking de EF Core (que sirve para detectar cambios y poder hacer `SaveChanges()`) sería trabajo de más para un escenario que jamás va a llamar a `SaveChanges()`. Es una optimización de performance justificada por el propósito real del servicio, no una moda.

Por qué en `Infrastructure/Persistence/` y no, por ejemplo, directamente en la raíz de `Infrastructure`: para dejar espacio a que la carpeta `Persistence` crezca de forma ordenada con sus propias subcarpetas (`Configurations/`, `Repositories/`) a medida que se agreguen más entidades, sin mezclar eso con otros posibles adaptadores de `Infrastructure` que no tengan que ver con persistencia (por ejemplo, un cliente HTTP hacia otro servicio).

Relación: depende de `Cliente` (Domain). Será usado por `ClienteConfiguration` (indirectamente, vía `OnModelCreating`) y directamente inyectado en `ClienteRepository` (Paso 3.3).

Paso 3.2 — El mapeo a la tabla real: `Infrastructure/Persistence/Configurations/ClienteConfiguration.cs`

Qué es: la clase `ClienteConfiguration : EntityTypeConfiguration<Cliente>` que mapea cada propiedad de `Cliente` a su columna real en `MPR_ASAS_MASEGURADO` (`Id` → `ASAS_NID`, `Nombre` → `ASAS_NOMBRE`, etc.), e ignora `NombreCompleto` porque no existe como columna física.

Por qué después del `DbContext` y no antes: `ApplyConfigurationsFromAssembly()` (dentro de `AppDbContext.OnModelCreating`) es lo que va a descubrir automáticamente esta clase — tiene sentido que el mecanismo que la va a usar ya esté declarado, aunque en la práctica ambos archivos podrían escribirse casi en simultáneo.

Por qué en un archivo de configuración separado (Fluent API) y no con Data Annotations sobre la propia entidad `Cliente`: porque poner atributos de EF Core (`[Column("ASAS_NID")]`, etc.) directamente sobre `Cliente` obligaría a Domain a referenciar `Microsoft.EntityFrameworkCore` — rompiendo la regla de que Domain no depende de nada externo (Paso 1.1). Al vivir este mapeo en `Infrastructure`, `Cliente` permanece “limpia” y solo `Infrastructure` sabe que existe una columna llamada `ASAS_NID`.

Por qué es crítico documentar bien esta clase: es el único lugar de todo el sistema donde el nombre de una columna física de una tabla que **no pertenece a este microservicio** queda registrado. Si esa tabla cambia de estructura en el sistema dueño, este es el único archivo que hay que tocar.

Relación: depende de `Cliente` (Domain) y de tipos de EF Core (`EntityTypeConfiguration<T>`, `ModelBuilder`). Es descubierta automáticamente por `AppDbContext` (Paso 3.1).

Paso 3.3 — El repositorio concreto: `Infrastructure/Persistence/Repositories/ClienteRepository.cs`

Qué es: la clase `ClienteRepository : IClienteRepository`, que recibe `AppDbContext` por constructor e implementa `ObtenerPorIdAsync` (con `FirstOrDefaultAsync`) y `ObtenerTodosAsync` (con `ToListAsync`).

Por qué es el último archivo de acceso a datos, y no el primero de esta capa: necesita que ya existan tanto la interfaz que va a implementar (`IClienteRepository`, Domain Paso 1.3) como el `DbContext` que va a usar internamente (Paso 3.1) y, en la práctica, que el mapeo (Paso 3.2) ya esté resuelto — de lo contrario, una consulta contra `_context.Clientes` fallaría en tiempo de ejecución al no saber a qué tabla/columnas corresponde.

Por qué esta es la única clase de todo el proyecto que “sabe” que existe EF Core y SQL Server, además del propio `DbContext`: es intencional — si mañana la fuente de este dato cambia (por ejemplo, a una llamada HTTP a otro microservicio dueño de los asegurados), **esta es la única clase que se reescribe**. Ni Domain, ni Application, ni Api se enteran del cambio, porque todos ellos solo conocen la interfaz `IClienteRepository`, nunca esta implementación.

Relación: implementa `IClienteRepository` (Domain); depende de `AppDbContext` (mismo proyecto) y de `Cliente` (Domain). Será registrada en el contenedor de DI en el siguiente paso, y consumida (a través de la interfaz) por los handlers de Application (Pasos 2.5 y 2.6) — sin que Application sepa que esta clase concreta existe.

Paso 3.4 — El registro en el contenedor de dependencias: `Infrastructure/DependencyInjection.cs`

Qué es: el método `AddInfrastructure(this IServiceCollection services, IConfiguration configuration)` que lee la cadena de conexión `ClientesDb` desde la configuración, registra `AppDbContext` con `UseSqlServer(connectionString)`, y registra `services.AddScoped<IClienteRepository, ClienteRepository>()`.

Por qué es el último archivo de esta capa: es aquí, y solo aquí, donde finalmente se conecta la interfaz (`IClienteRepository`), de

Domain) con su implementación real (`ClienteRepository` , de esta misma capa). Antes de este archivo, ambas piezas existen pero nada las une en tiempo de ejecución.

Por qué recibe `IConfiguration` en vez de tener la cadena de conexión hardcodeda dentro del método: para que la cadena de conexión sea un detalle de configuración del entorno (`appsettings.json` , variables de entorno, User Secrets — nunca código fuente), consistente con la buena práctica de no mezclar configuración con lógica. (Nota: como se documentó en el análisis anterior, hoy esa buena práctica de patrón se ve opacada porque el *valor* de la cadena de conexión sí quedó hardcodedo en `appsettings.Development.json` — el patrón de diseño de este archivo está bien, el problema es dónde se guardó el valor real).

Relación: conecta `IClienteRepository` (Domain) con `ClienteRepository` (misma capa) y con `AppDbContext` (misma capa). Será invocado desde `Program.cs` (Paso 4.1), que es quien le pasa `builder.Configuration` .

CAPA 4 — API (el punto de entrada HTTP)

Carpeta: `src/Vitamedica.Microservicios.Clientes.Api/`

Ya existen las tres capas internas completas y funcionando entre sí (aunque todavía inalcanzables desde fuera del proceso). Ahora se construye la única capa que expone todo esto por HTTP.

Paso 4.1 — El composition root: `Api/Program.cs`

Qué es: el archivo que crea el `WebApplicationBuilder` , registra `AddControllers()` , `AddEndpointsApiExplorer()` , `AddSwaggerGen()` , llama a `builder.Services.AddApplication()` y `builder.Services.AddInfrastructure(builder.Configuration)` , configura Swagger solo en Development, y arma el pipeline HTTP (`UseHttpsRedirection` , `UseAuthorization` , `MapControllers`).

Por qué se escribe antes que el Controller y no después: `Program.cs` es lo que arranca el proceso y “conecta los cables” entre todas las capas — necesita existir para que, cuando se agregue el primer Controller, la aplicación ya sepa resolver sus dependencias (los handlers de Application, que a su vez necesitan `IClienteRepository` resuelto por Infrastructure). Sin este archivo, un Controller con dependencias inyectadas no tendría quién se las provea.

Por qué es el único lugar de todo el proyecto que conoce (mediante `using`) tanto a `Application` como a `Infrastructure` : porque es el composition root — el único punto legítimo donde está bien “conocer todo”, ya que su única responsabilidad es ensamblar el sistema completo antes de que arranque. Ningún otro archivo del proyecto debería tener este nivel de conocimiento cruzado entre capas.

Relación: invoca `AddApplication()` (Paso 2.7) y `AddInfrastructure()` (Paso 3.4). A partir de aquí, el contenedor de DI ya sabe resolver `ObtenerClientePorIdQueryHandler` , `ListarClientesQueryHandler` (con su `IClienteRepository` resuelto internamente a `ClienteRepository`).

Paso 4.2 — El controller: `Api/Controllers/ClientesController.cs`

Qué es: la clase `ClientesController : ControllerBase` , con `[ApiController]` y `[Route("api/[controller]")]` , que recibe `ObtenerClientePorIdQueryHandler` y `ListarClientesQueryHandler` por constructor, y expone `GET /api/clientes` (método `Listar`) y `GET /api/clientes/{id:int}` (método `ObtenerPorId`).

Por qué es el último archivo de todo el flujo: es el único punto que finalmente queda alcanzable desde fuera del proceso (por HTTP). Todo lo anterior existía y compilaba, pero nada podía todavía “dispararse” desde una petición externa hasta este momento.

Por qué es tan “delgado” (arma la Query, llama al handler, traduce a `ActionResult` , y nada más): porque toda la lógica real (qué hacer con la petición) ya vive en Application. El Controller no debe tomar ninguna decisión de negocio — solo traducir entre el mundo HTTP (rutas, verbos, códigos de estado) y el mundo de los casos de uso (`Query` → `Result` /DTO). Esto es lo que permite, por ejemplo, escribir un test de `ObtenerClientePorIdQueryHandler` sin necesitar levantar un servidor HTTP.

Relación: depende de `ObtenerClientePorIdQueryHandler` y `ListarClientesQueryHandler` (Application) — no de sus implementaciones internas, solo los invoca por su firma pública. Es resuelto por el contenedor de DI configurado en `Program.cs` (Paso 4.1).

Paso 4.3 — La configuración: `Api/appsettings.json` y `Api/appsettings.Development.json`

Qué es: los archivos con la sección `ConnectionStrings:ClientesDb` que `AddInfrastructure` (Paso 3.4) necesita leer para poder construir el `AppDbContext` .

Por qué es, en la práctica, uno de los últimos pasos, aunque técnicamente podría haberse hecho antes: hasta este punto todo el código ya compila sin necesidad de una cadena de conexión real — el error de “falta configurar ‘ClientesDb’” (el `throw` en `AddInfrastructure`) solo aparece en tiempo de **ejecución**, al arrancar la app. Por eso, en un desarrollo real, este es el primer momento en que hace falta un valor real para poder efectivamente correr el servicio y probarlo end-to-end vía Swagger.

Advertencia de seguridad para este paso, aprendida del análisis anterior: el valor de la cadena de conexión (usuario, contraseña, IP del servidor) **no debe** escribirse en `appsettings.Development.json` si ese archivo se va a versionar en git. Debe ir a User Secrets (`dotnet user-secrets set "ConnectionStrings:ClientesDb" "..."`) en desarrollo local, o a variables de entorno / un vault en cualquier entorno

compartido.

Relación: es leído en tiempo de ejecución por `AddInfrastructure()` (Paso 3.4), a través de `IConfiguration` .

RESUMEN VISUAL DEL ORDEN COMPLETO

```
1. Vitamedica.Microservicios.Clientes.slnx
2-5. Los 4 .csproj de src/ (Domain → Application → Infrastructure → Api), con sus ProjectReference ya declaradas

DOMAIN (no depende de nada)
6. Domain/Entities/Cliente.cs
7. Domain/Exceptions/DomainException.cs
8. Domain/Repositories/IClienteRepository.cs
9. tests/.../ClienteTests.cs (+ InternalsVisibleTo en Domain.csproj)

APPLICATION (depende de Domain)
10. Application/Dtos/ClienteDto.cs
11. Application/Clientes/ClienteMapper.cs
12. Application/Common/Abstractions/IQueryHandler.cs
13. Application/Common/Abstractions/Result.cs
14. Application/Clientes/Queries/ListarClientesQuery.cs
15. Application/Clientes/Queries/ObtenerClientePorIdQuery.cs
16. Application/Clientes/Queries/ListarClientesQueryHandler.cs
17. Application/Clientes/Queries/ObtenerClientePorIdQueryHandler.cs
18. Application/DependencyInjection.cs

INFRASTRUCTURE (depende de Domain y Application)
19. Infrastructure/Persistence/AppDbContext.cs
20. Infrastructure/Persistence/Configurations/ClienteConfiguration.cs
21. Infrastructure/Persistence/Repositories/ClienteRepository.cs
22. Infrastructure/DependencyInjection.cs

API (depende de Application e Infrastructure)
23. Api/Program.cs
24. Api/Controllers/ClientesController.cs
25. Api/appsettings.json + Api/appsettings.Development.json (con la cadena de conexión, NUNCA hardcodeada en texto plano si se versio
```

Idea para llevarte de aquí: en cualquier funcionalidad nueva que construyas sobre esta arquitectura, el orden va a ser siempre el mismo patrón — Entidad → Interfaz de repositorio → DTO → Mapper → Query/Command → Handler → Registro DI de Application → DbContext/Configuration → Repositorio concreto → Registro DI de Infrastructure → Controller/endpoint → Configuración. No memorices la lista de 25 pasos de este ejemplo puntual; memoriza la regla de una sola línea del principio: **de adentro (lo que no depende de nada) hacia afuera (lo que depende de todo).**